

# Enterprise AI Pilot Planning Assistant Final Report

## Project Outcome

I built a public Streamlit application that converts a bounded, low-risk SME scenario into a structured AI pilot-plan draft. It requires a named test group, dated stages, a metric with a baseline, a feedback instrument, risks with mitigations, and a human approval checkpoint. My ten-case evaluation found a mean score of 5.0/5 for the structured workflow and 3.1/5 for a direct raw-model request, an improvement of 1.9 points. This finding covers explicit components only; it does not establish factual correctness, feasibility, safety, or business value.

## Problem and Product Decision

The primary user is an SME manager or project owner who understands the business problem but lacks a dedicated AI product or evaluation team. That person needs a draft that colleagues can challenge before the company commits resources or exposes real data. A static template can require headings, but it cannot adapt its stages, metric, feedback method, and risks to varied situations. A direct foundation-model request is flexible, yet may omit required details.

I therefore defined a narrow product claim: a form plus a fixed system prompt should improve the explicit coverage of five required pilot-plan components compared with asking the same model directly. The application does not claim to replace business judgment. Every result is labelled as a draft, and a project owner must approve or reject it before implementation or expansion.

## System Design and Tradeoffs

The smallest working version follows one path: one form submission, one hosted-model call, one JSON response, and one human decision. Streamlit collects seven required inputs: company type, use case, participant role and number, duration, start date, pain point, and constraint. Python validates the fields, limits duration to 1-12 weeks, calculates the end date, builds the prompt, and rejects a response that is not a JSON object.

I use a rented foundation model because the task is conditional text generation across varied contexts. Narrow machine learning would need labelled examples for a fixed prediction target. Retrieval is unnecessary because this version does not answer from company documents, while an agent would add a multi-step tool loop that the workflow does not need. I own the interface, validation, prompt, schema, parser, and evaluation; OpenRouter provides access to openai/gpt-4.1-mini.

The model proposes the objective, schedule, metric, feedback process, and mitigations. It does not perform arithmetic, store submissions, or authorise action. The interface warns users not to enter personal data, client names, credentials, financial details, contracts, or confidential strategy. Health, credit, hiring, legal, security, and other high-stakes decisions are out of scope.

## Data and Evaluation Method

I froze ten fictional SME cases before comparing outputs. The dataset covers ten industries and low-risk tasks such as drafting, summarising, classifying public text, and preparing surveys. Each row records the company context, participants, two- or three-week duration, pain point, constraint, and the common start date of 12 October 2026. The cases contain no real organisations, personal identifiers, credentials, contracts, or confidential data. The generation specification is committed with the CSV.

The experiment held the cases, model, provider route, temperature of 0.2, and scoring rules constant. The structured condition used validated fields, a fixed prompt, JSON mode, and a schema. The baseline used one user message with the same case information but no system prompt or required format. The ten cases produced 20 plans and the same model made 20 judging calls, for 40 API calls in total.

Before running the comparison, I set two targets: a structured mean of at least 4.0/5 and an improvement of at least 1.0 point over the raw baseline. Each plan received zero or one for: a named test group; at least two dated or numbered stages; a numeric target plus baseline or comparison; a named feedback instrument plus respondent or timing; and explicit human approval before rollout. The rubric instructs the reviewer to score only text that is present and not infer missing details.

## Results and Evaluation Critique

| Measure                         | Structured workflow | Raw baseline | Difference   |
|---------------------------------|---------------------|--------------|--------------|
| Mean five-item score            | 5.0/5               | 3.1/5        | +1.9         |
| Mean generation latency         | 5.5 seconds         | 7.2 seconds  | -1.7 seconds |
| Generation cost for ten outputs | US\$0.0085          | US\$0.0128   | -US\$0.0042  |

Both targets were met. The structured plans achieved 50 passes out of 50 checks; the raw plans achieved 31. Raw outputs passed the test-group and dated-stage criteria in every case, so the structure added no measured value on those easier items. Raw pass rates were 40% for metrics, 40% for feedback instruments, and 30% for approval checkpoints, compared with 100% for structured outputs.

Latency and cost were monitored rather than used as pass or fail criteria. In this run, structured generation was faster and cheaper, but one run cannot establish a stable performance advantage. Provider prices and hosted models may change, and repeated generations may differ.

The evaluation exposed a weakness in model-based judging. The automated judge reported 5.0/5 for structured plans and 4.0/5 for raw plans. Manual adjudication reduced the raw mean to 3.1/5 because the judge accepted vague feedback, targets without baselines, and review that did not gate rollout. I preserved the original evidence and recorded corrections separately. The result is auditable, but not independent because I built the system and reviewed the scores.

## **Engineering Changes and Remaining Rough Edges**

The most important revision followed the instructor's feedback that “reliable and comprehensive” could not be scored. I replaced that phrase with five explicit binary checks and compared the system against the same ten raw-model cases. I also tightened the system prompt so it requires date-bounded stages, a numeric target with a baseline, a specific feedback instrument, and approval before implementation. The user context is delimited and treated as data, JSON mode is requested, temperature is set to 0.2, and invalid JSON raises an error instead of being silently displayed.

These controls improve consistency, but they do not verify that a target is realistic or a mitigation will work. The provider has a 60-second timeout but no retry or backoff policy. The application also lacks authentication, a database, accessibility testing, document upload, retrieval, and automatic correction. A well-formatted plan may still contain poor advice, so the human checkpoint is a required control.

## **Limitations and Future Path**

The ten cases are synthetic, English-only, small, and intentionally clean. They do not represent incomplete requirements, multilingual users, hostile prompts, accessibility needs, organisational politics, or messy data. One model and one run per condition cannot measure run-to-run variance or transfer. Because the prompt directly requests the scored items, the perfect score mainly demonstrates instruction adherence. It does not measure accuracy, feasibility, satisfaction, wider safety, or realised value. Delimiting input cannot eliminate prompt injection.

The next evidence milestone should be a usability study with SME managers and two independent reviewers. I would measure time to first usable draft, edit distance, perceived usefulness, critical-error rate, inter-rater agreement, and stability across repeated runs and models. A blinded second scorer would directly address the current evaluation weakness. Product features should expand only if that evidence supports them; the immediate priority is stronger validation, not more automation.

## **Conclusion**

The project changed a vague direct-model request into a bounded and auditable planning workflow. On the frozen ten-case test, the structured approach improved explicit component coverage by 1.9 points while keeping the decision with a human owner. The result supports the narrow project claim, while the saved data, rubric, outputs, corrections, code, public application, and recorded demonstration make that claim inspectable.